

UNIT 5 – Django REST Framework, APIs & React Integration

Exhaustive Study Notes covering REST Principles, Serializers, DRF Views, JWT Auth, CORS & React Integration

1. API

An Application Programming Interface (API) defines contracts for two software systems to communicate over network protocols (HTTP).

2. REST API

Representational State Transfer (REST) is an architectural style utilizing standard HTTP methods: GET (Retrieve), POST (Create), PUT (Replace), PATCH (Partial Update), DELETE (Remove).

3. Django REST Framework

DRF is a powerful toolkit for building Web APIs in Django featuring Serializers, WebBrowsable API, Authentication, and Permissions.

4. Add DRF

Install via `pip install djangorestframework` and add `'rest_framework'` to `INSTALLED_APPS` in `settings.py`.

5. Serializer

Serializers convert complex Python datatypes and QuerySets into JSON format (Serialization) and parse JSON payload into validated Python objects (Deserialization).

6. ModelSerializer

A `ModelSerializer` automatically generates serializer fields from a Django Model:

```
from rest_framework import serializers
from .models import Student

class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = '__all__'
```

DRF Serialization & Deserialization Processing Pipeline

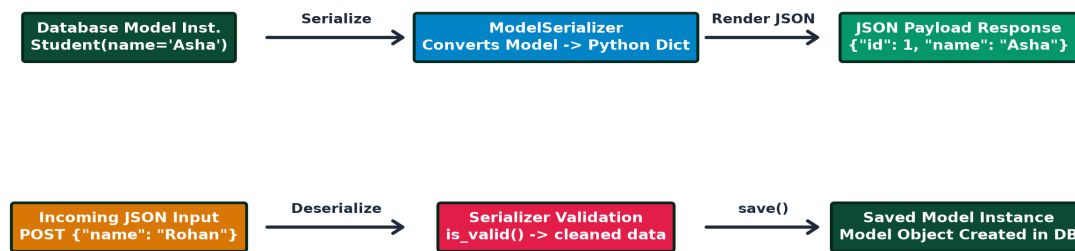


Diagram: DRF Serialization & Deserialization Processing Pipeline.

7. Request and Response

DRF extends Django's request with `Request` (parsing `request.data`) and provides `Response` (rendering Python dict to JSON).

8. @api_view

Decorator `@api\_view(['GET', 'POST'])` turns function-based views into DRF API views.

9. GET API

Retrieves objects and returns serialized JSON array or object.

10. POST API

Accepts JSON payload, validates via `serializer.is\_valid()`, saves data, and returns 201 Created.

11. PUT API

Performs complete resource replacement update.

12. PATCH API

Performs partial resource field update using `partial=True`.

13. DELETE API

Deletes target resource and returns status 204 No Content.

14. HTTP Status Codes

200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error.

15. API URL Routing

Connecting DRF views to URL patterns in `urls.py`.

16. CRUD API

Complete set of RESTful API endpoints covering List, Create, Detail, Update, and Delete operations.

17. API Authentication

Methods to identify API request senders: SessionAuth, TokenAuth, JWTAuth.

18. JWT Authentication

JSON Web Token authentication (`django-rest-framework-simplejwt`) issuing signed Access Tokens (short-lived) and Refresh Tokens (long-lived).

JWT (JSON Web Token) Authentication & Refresh Workflow

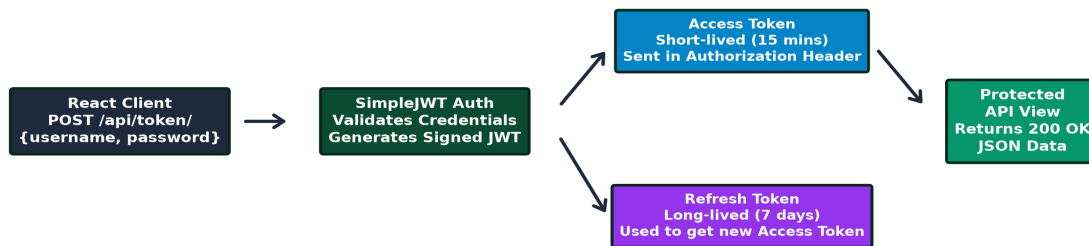


Diagram: JWT (JSON Web Token) Authentication & Refresh Token Flow.

19. API Permissions

Restricts API access using policies like `AllowAny`, `IsAuthenticated`, `IsAdminUser`, or custom permissions.

20. CORS

Cross-Origin Resource Sharing permits browsers to allow React frontend (e.g. localhost:3000) to call Django backend API (localhost:8000). Handled via `django-cors-headers`.

Decoupled React SPA + Django REST Framework Architecture



Diagram: Decoupled React SPA + Django REST Framework Architecture.

21. React + Django

Decoupled architecture where React handles frontend UI state and Django DRF serves JSON APIs.

22. Axios

Promise-based JS HTTP client used in React to consume Django APIs (`axios.get()`, `axios.post()`).

23. Authorization Header

Frontends send JWT tokens in request headers: `Authorization: Bearer <access_token>`.

24. Postman

Desktop tool to test, debug, and document REST API endpoints.

25. API Validation

Serializer validation rules (`validate_<fieldname>()` and `validate()`).

26. API Error Handling

Standardized JSON error responses with clear status codes.

27. Function-Based vs Class-Based API Views

Comparing `@api_view` FBVs vs `APIView` class inheritance.

28. Generic API Views

Pre-built DRF views like `ListCreateAPIView` and `RetrieveUpdateDestroyAPIView`.

29. ViewSets and Routers

`ModelViewSet` groups all CRUD logic into a single class, and `DefaultRouter` automatically generates REST URL patterns.

30. API Security Basics

Input validation, Throttling/Rate limiting, HTTPS enforcement, and secure token handling.